PRACTICAL - 3

Aim:

To develop Java programs that demonstrate the use of classes, constructors, and method overriding for object comparison.

1. In the first program, a Point class with integer coordinates is created, overriding toString(), equals(), and hashCode(). A driver class builds an array of points with repeated coordinates and counts the number of distinct points.
2. In the second program, a Card class with rank and suit is created, overriding toString(), equals(), and hashCode(). A driver class inputs cards into an array and detects the first duplicate card using the overridden equals() method.

Code Screenshot:
Part a:

simple.java largest.java test.java Card.java Driver.java Customer.java

test.java > test > main(String[])

```java
import java.util.Scanner;

public class test {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print(s: "Enter number of points: ");
        int n = sc.nextInt();

        Point[] a = new Point[n];


        for (int i = 0; i < n; i++) {
            System.out.print("Enter x and y for point " + (i+1) + ": ");
            int x = sc.nextInt();
            int y = sc.nextInt();
            a[i] = new Point(x, y);
        }

        int distinctCount = 0;

        for (int i = 0; i < n; i++) {
            int j;
            for (j = 0; j < i; j++) {
                if (a[i].equals(a[j])) {
                    break;
                }
            }
            if (j == i) {
                distinctCount++;
            }
        }

        System.out.println("Distinct: " + distinctCount);
    }
}

```

Point.java > ...

```java
import java.util.Objects;

class Point {
    private int x;
    private int y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    @Override
    public String toString() {
        return "(" + x + ", " + y + ")";
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Point other = (Point) obj;
        return this.x == other.x && this.y == other.y;
    }

    @Override
    public int hashCode() {
        return Objects.hash(x, y);
    }
}
```

Part b :

Driver.java > Driver > main(String[])

```java
import java.util.Scanner;

public class Driver {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print(s: "Enter number of cards: ");
        int n = sc.nextInt();
        sc.nextLine();

        Card[] cards = new Card[n];

        for (int i = 0; i < n; i++) {
            System.out.print("Enter rank and suit for card " + (i+1) + ": ");
            String rank = sc.next();
            String suit = sc.next();
            Card newCard = new Card(rank, suit);


            for (int j = 0; j < i; j++) {
                if (newCard.equals(cards[j])) {
                    System.out.println("Duplicate found: " + newCard);
                    return;
                }
            }

            cards[i] = newCard;
        }
    }
}
```

Card.java > Card

```java
import java.util.Objects;

class Card {
    private String rank;
    private String suit;

    // Constructor to initialize rank and suit
    public Card(String rank, String suit) {
        this.rank = rank;
        this.suit = suit;
    }


    @Override
    public String toString() {
        return rank + " of " + suit;
    }


    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Card other = (Card) obj;
        return this.rank.equals(other.rank) && this.suit.equals(other.suit);
    }


    @Override
    public int hashCode() {
        return Objects.hash(rank, suit);
    }
}

```

Output Screenshot:

Part a :

```
Enter rank and suit for card 1: ace of spades
Enter rank and suit for card 2: king diamon
PS E:\java> cd "e:\java\" ; if ($?) { javac test.java } ; if ($?) { java test }
Enter number of points: 5
Enter x and y for point 1: 2 3
Enter x and y for point 2: 2 3
Enter x and y for point 3: 2 3
Enter x and y for point 4: 2 3
Enter x and y for point 5: 2 3
Distinct: 1
PS E:\java>
```

Part b :

```
PROBLEMS 8   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS E:\java> cd "e:\java\" ; if ($?) { javac Driver.java } ; if ($?) { java Driver }
Enter number of cards: 3
Enter rank and suit for card 1: king diamond
Enter rank and suit for card 2: queen diamond
Enter rank and suit for card 3: king diamond
Duplicate found: king of diamond
PS E:\java>
```

Learning Outcome:

In this practical, we learned how to create Java classes with private data members and constructors to represent objects. We practiced overriding the toString() method to display object information in a readable format, and overriding the equals() and hashCode() methods to compare objects based on their data instead of memory addresses. By storing objects in arrays, we applied object comparison to count distinct points and detect duplicate cards. This helped us understand the difference between reference equality (==) and logical equality (equals()), and showed how to implement simple logic for identifying unique or repeated objects without using Java Collections.